

Lab 12

① `#!/bin/bash`

`n=10`

`for ((i=0; i  $\leq$  n; i++))`

`do`

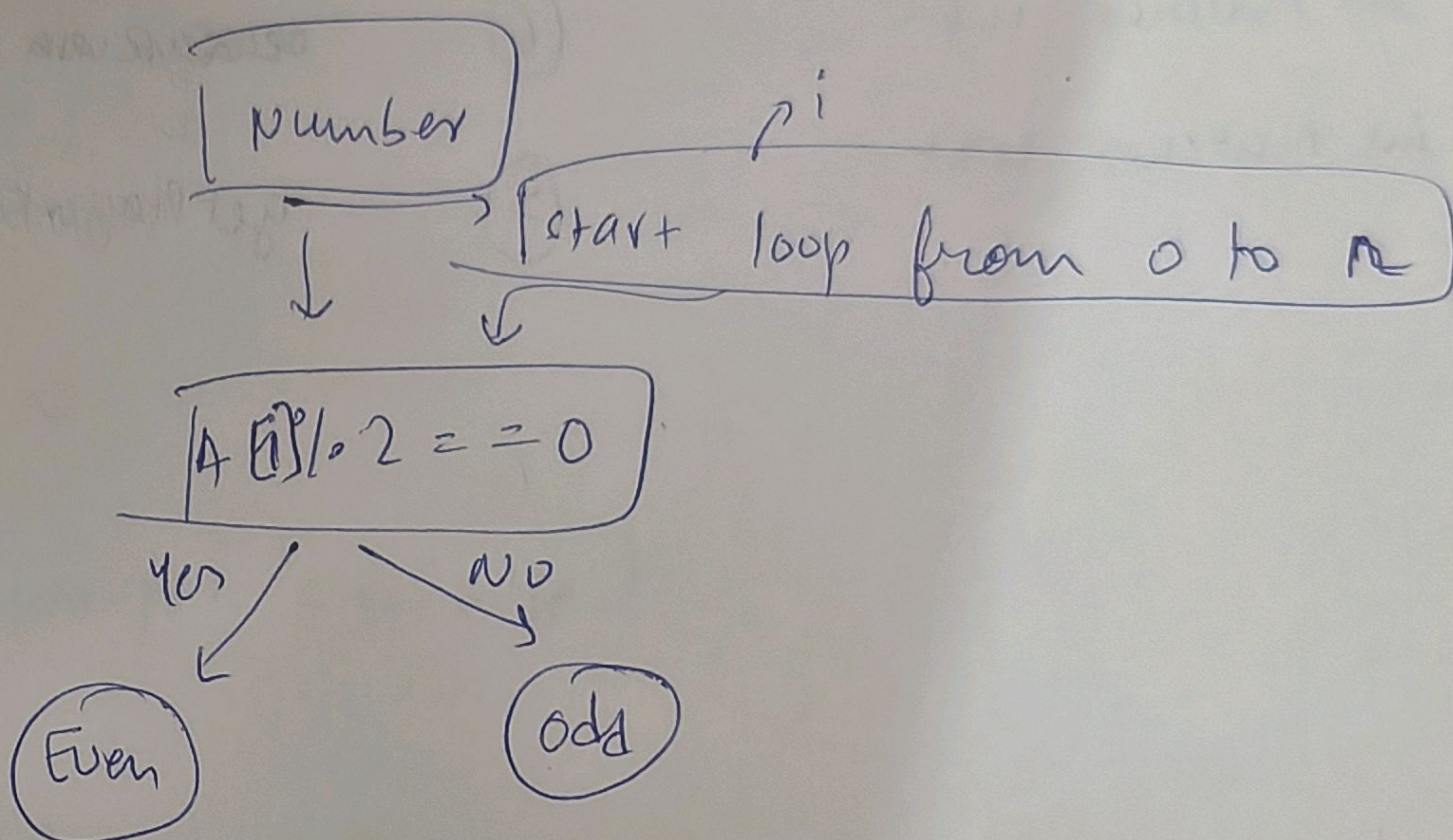
`echo "i" (i)`

`done`

Take a variable `n`

start the loop from `i` to `n` and print `i`.

②



`echo "Enter a number:"`

`read n`

`for ((i=0; i  $\leq$  n; i++))`

`do`
`{`

`if [  $i \% 2 \neq 0$  ]; then`

`{`

`echo "Number $i is even"`

`}`

`else`

`{`

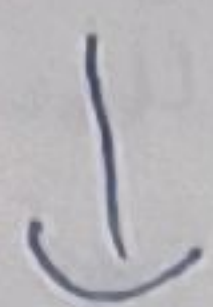
`echo "Number $i is odd"`

`}`

`done`

3

Take user input n



start loop & put a condition

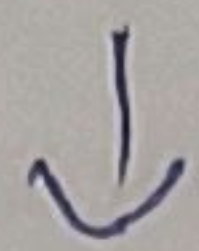
until it is > the ~~value~~ 15

(if it goes above 15 loop will

stop)

↓ initial i = 1

start loop from 1 to 15



print "number @ x i = \$(n * i)"

eg $2 \times 2 = 4$ $2 * 2$

echo "Enter a number"

read n

~~set~~ i = 1

-gt

until (~~n~~ i > 15)

do

✓

echo " \$n * \$i = \$(n * i) "

}

done

~~done~~

④

read -ra numbers < num.txt

read the
file raw
(as it
is)
two file
input
into
array

~~array~~ This is
to store the variable
in which
we have to
save the numbers

This is the file which has
numbers

Numbers $\rightarrow [2, 3, 5, 7]$

Take out length $\rightarrow$ $\downarrow$
of array start

↓
for every i print
↓

or +
 $A[i] * A[i]$
↓
to Double

read -rc numbers < num.txt

$$A \approx \{ \text{numbers } [^*] \}$$
$$l = \{ \text{numbers } [a] \}$$

for $((i \geq 0 \wedge j \leq 0 \wedge i + j = 1))$

do

return \$ (1 * numbers [i] + numbers [i]))

done

5

Take 2 numbers as arguments

using \$1 & \$2 → in a & b variable

↓

~~check~~ Put conditions

↓

IF \$2 -eq 0

yes

no

quotient → a/b

remainder a%b

↓

scale to 2

cannot divide
by 0

function divide {

local a=\$1

local b=\$2

if [\$b -eq 0]; then

echo "cannot divide by zero"

return

fi

local q=\$(echo "scale=2; \$a/\$b" | bc)

local r=\$(echo "scale=2; \$a%\$b" | bc)

echo "Quotient is \$q"

echo "Remainder is \$r"

}

divide

6

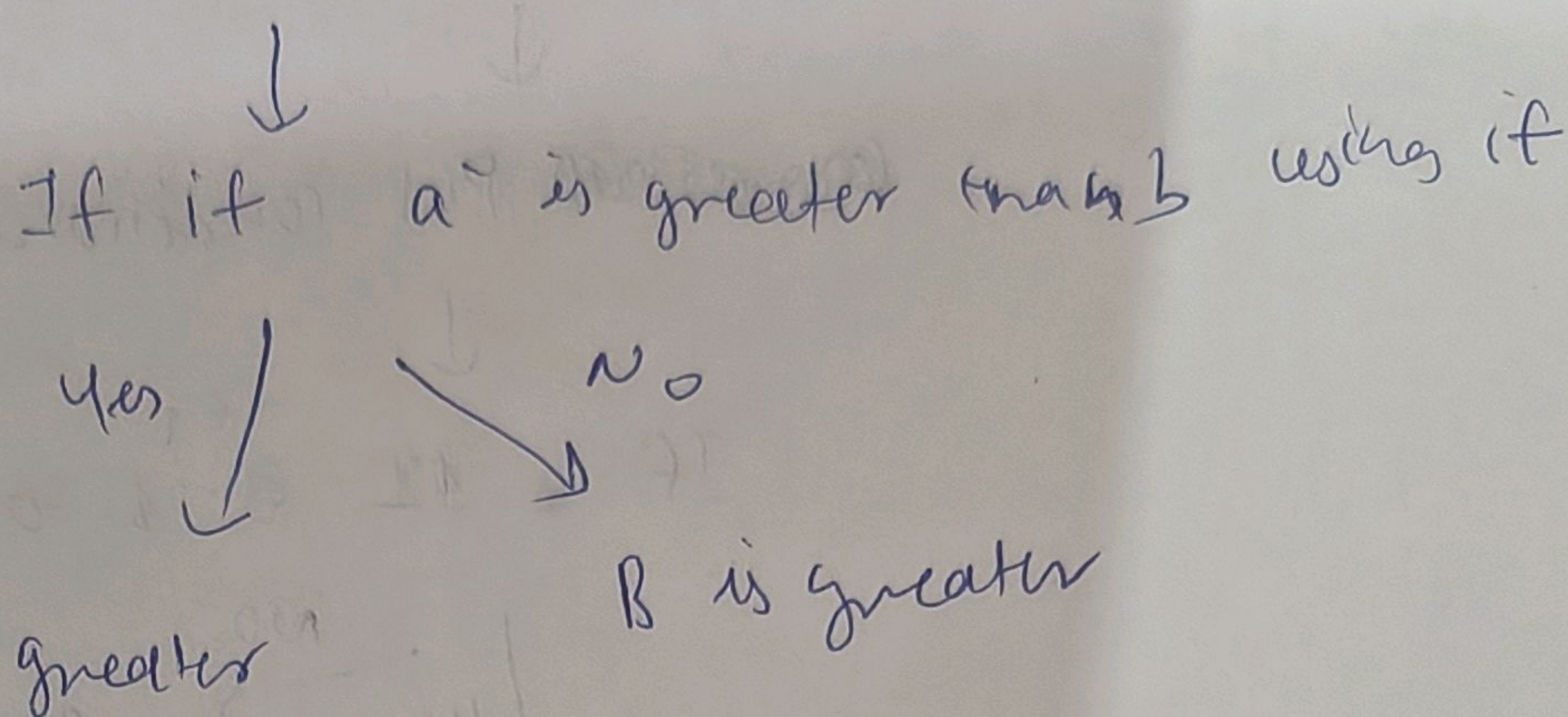
3

par

give arguments

6)

Take 2 number using argument
and store in variable a & b



function maximum {

a = \$1

b = \$2

if [\$a -gt \$b]; then

{

echo "\$a is greater"

}

else

~~echo~~ echo "\$b is greater"

fi

}

maximum 7 8

← pass arguments.

7) ~~Program~~

Create a function direxists

↓ → Take input from user for ~~filename~~
using -e check your directory name
↑
which is
for checking
if dir exists or not

yes / no
↓ ↓
Directory is to exist list down Directory doesn't exist
mkdir new filename
↑
To make new directory

function direxists {

echo "Enter filename"

read file

if [-e \$file]; then

<

echo "Directory exists"

ls \$file

}

else

mkdir newdir

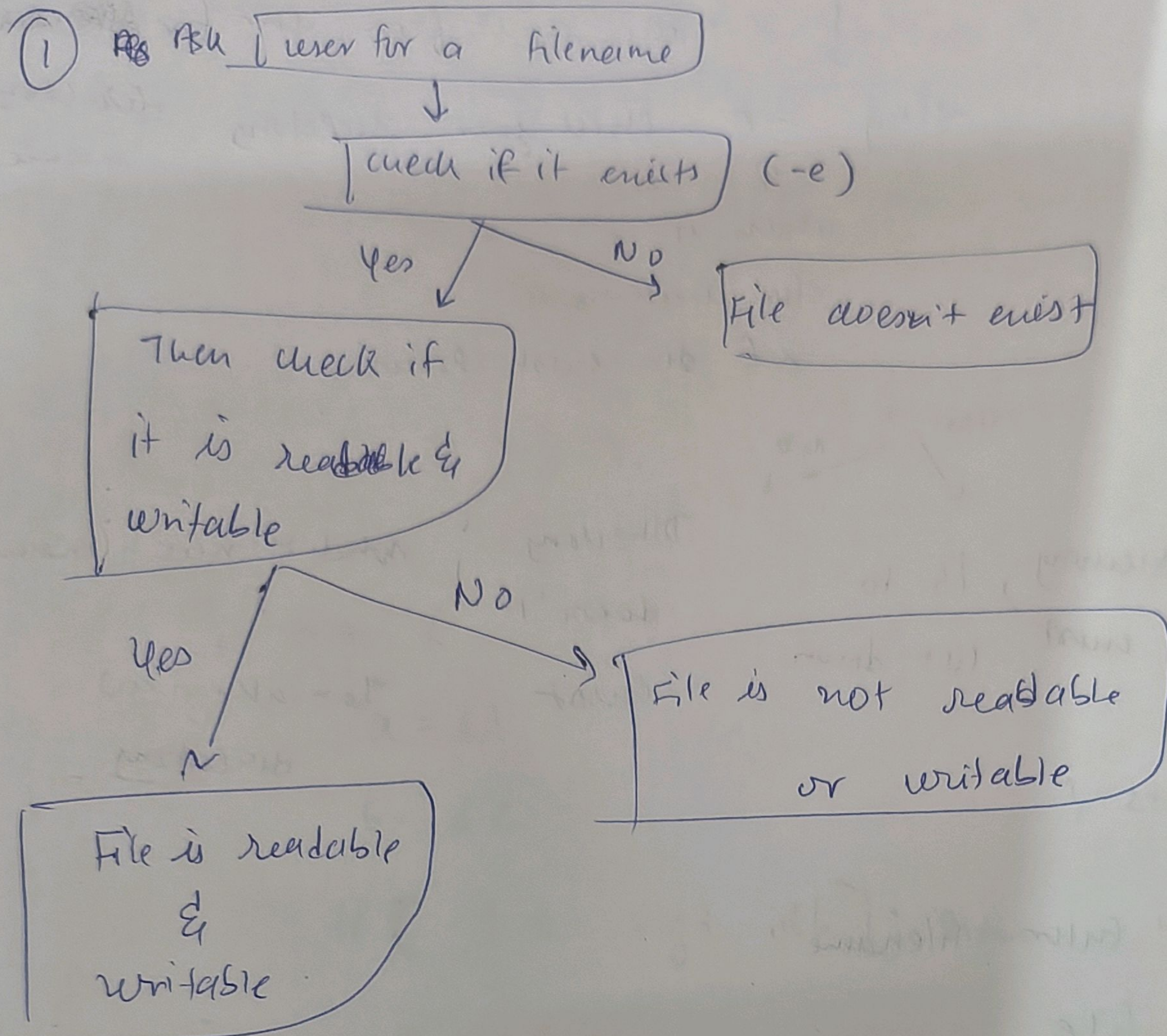
echo "New Directory is created"

fi

}

dir exists.

18th Aug



echo "Enter a filename"

read file

if [-e \$file]; then

if [-w \$file && -r \$file]; then

{

echo "File exist & is readable & writable"

}

else

echo "File exist but
File is not readable or writable"

fi

else

echo "File does not exist"

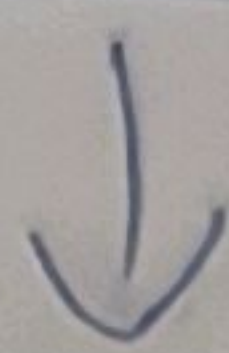
fi

②

Take a file ^{input} from user ~~input~~



read the file



Count the character using

wc -w words

-l lines

-m characters

echo "Enter filename"

read file

lines = \$(wc -l < \$file)

words = \$(wc -w < \$file)

characters = \$(wc -m < \$file)

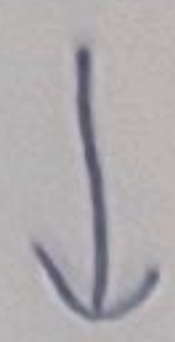
echo "No. of lines:" \$lines

echo "No. of words:" \$words

echo "No. of characters:" \$characters

3

List the ^{contents of} ~~the~~ current directory
using `ls cd`



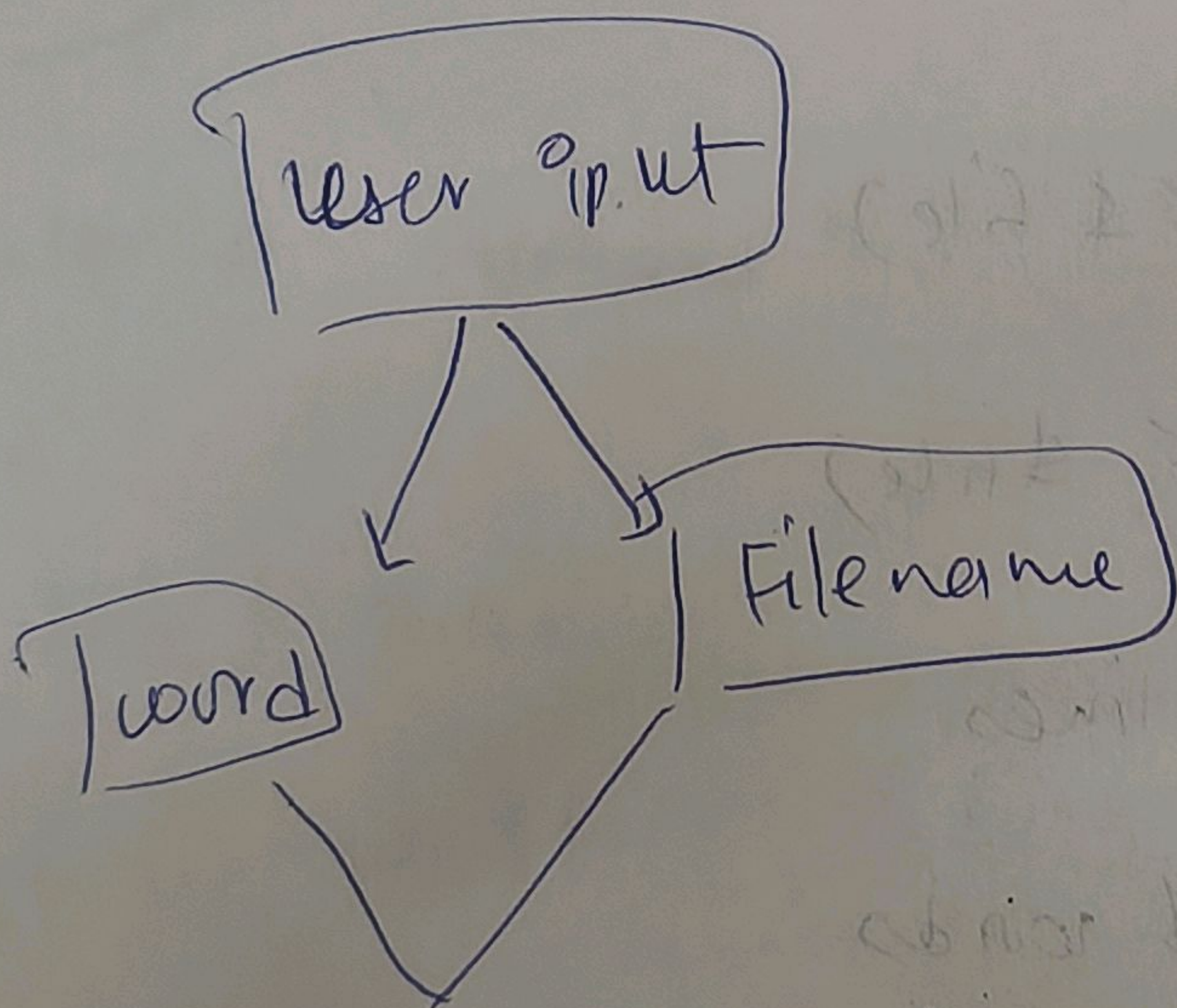
Then pipe the `ls -l` output &
find ^{largest filename} ~~the~~ and size using
`head -n 1`

```
echo "Contents of current dir"  
ls
```

```
largest file name and size = $(ls -l | head -n 1)
```

```
echo "Largest file name & size is: " $largest filename and  
size.
```

4



`grep [find] the $word from the
file entered by the user.`

```
echo "Enter the word"
```

```
read word
```

```
echo "Enter the filename:"
```

```
read file
```

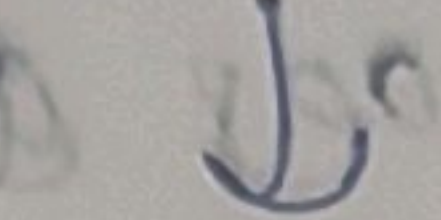
```
grep "$word" "$file"
```

5

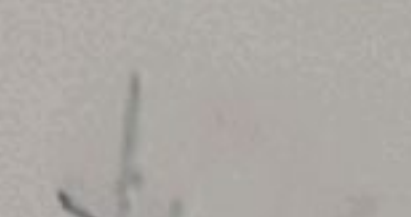
Take a number input
from user



Run a loop initializing
i from 1 to 20



print number i inside the loop



echo "Enter a no."

read n

for((i=1; i<=20; i++))

do

echo -n "x \$i = \$((\$n * \$i)) "

done

6

Get current disk usage
using `df`



Check if usage is above 80%

yes

no

Print file is safe.

Print

warning
message

happy number

Take n input from user

↓

initialise $sum = 0$

↓

Run loop ~~do~~ while $n \neq 0$

Compute this to take out

digits of the number

↓

$digit = n \% 10 \rightarrow$ remainder

$sum = sum + digit * digit$

$n = n / 10 \rightarrow$ to reduce one

digit for next

loop

Eg $121 \rightarrow 12$

↓

then $n = sum$

↓

If $n = 1$?

No

Yes

Not a

happy
number

happy number

happy ()

{

n = \$1

while (\$n -ne 1)

do

sum = 0

while [\$n -gt 0]

do

digit = \$((n % 10))

sum = \$((sum + digit * digit))

n = \$((n / 10))

done

n = \$sum

if [\$n -eq 1], then

{ echo "happy no."

}

else

echo "unhappy no."

return

fi

done

}

happy

22